LangChain Basics

Build your first AI application — step by step

Using

Google colab

AI Product Management Cohort • Ashu Mishra

What We'll Cover Today

1. **What is LangChain?**
   Why it exists & the big picture
2. **Calling the AI Model**
   Talk to GPT using Python
3. **Prompt Templates**
   Reusable instructions for AI
4. **Chains**
   The assembly line pattern
5. **Output Parsers**
   Get clean, structured answers
6. **Exercise**
   Build your own chain!

What is LangChain?

Think of it as a toolkit that helps you build apps on top of AI models

🍕 **Building an app with AI is like running a restaurant.**

You have a great chef (the AI model). But you also need a menu (prompts), a waiter (chains), and a delivery system (output). LangChain gives you all of that infrastructure — ready to use.

🤖

AI Models

GPT, Gemini, Claude — the brain

🔧

LangChain

The toolkit that connects everything

🚀

Your App

The product your users actually see

What is LangChain?

- What is LangChain?
  - A framework for building applications with large language models (LLMs)
  - Allows you to chain together different components to create more complex and powerful applications
- Key features
  - Composable: You can easily combine different LLMs, prompt templates, and agents to create custom applications
  - Flexible: LangChain can be used for a variety of tasks, including chatbots, question answering, summarization, and more
  - Extensible: LangChain is open source and extensible, so you can add your own custom components

Composable

You can easily combine different LLMs, prompt templates, and agents to create custom applications.

Flexible

LangChain can be used for a variety of tasks, including chatbots, question answering, summarization, and more.

Extensible

LangChain is open source and extensible, so you can add your own custom components.

Part 1: Talking to the AI Model

LLM = Large Language Model — it's the brain behind ChatGPT

💡 Analogy: Imagine a brilliant assistant locked in a room. You slide a note through a slot. They read it, write a reply, and slide it back. That's an LLM!

You write a message → Python sends it to AI → AI reads & thinks → AI sends reply back → You see the answer

```
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)

response = llm.invoke("Explain LangChain in 2 sentences.")

print(response.content)
```

What is Temperature?

A creativity dial for your AI — from robot-precise to wildly creative

🎯

`temp = 0.0`

Very Predictable

Always gives the same answer. Use for maths, facts, data.

⚖️

`temp = 0.7`

Balanced

Creative but sensible. Best for most everyday tasks.

🎨

`temp = 1.0`

Very Creative

Unpredictable and imaginative. Use for stories, brainstorming.

THE RESTAURANT RECOMMENDATION ANALOGY

0.0

Safe & reliable

"Which McDonald's is closest to me?"

Same answer every time. No surprises. Just facts.

0.7

Open to suggestions

"Suggest a good place for dinner tonight."

Helpful and sensible. A bit of flavour, not robotic.

1.0

Feeling adventurous

"Surprise me. Something I've never tried."

Maximum creativity. Could be amazing. Could be wild.

THE PRODUCT MANAGER'S DECISION

LEGAL / COMPLIANCE

Document checker

`temp = 0.0`

Consistency is everything. Same input must give same output.

CUSTOMER SUPPORT

Support chatbot

`temp = 0.5`

Friendly and natural, but never unreliable or off-script.

MARKETING

Tagline generator

`temp = 0.9`

Go wild. Creativity is the whole point here.

> Setting temperature is a **product decision** — it defines how your AI behaves for your users, every single time.

GUIDE TO TEMPERATURE & AGENTIC AI SYSTEMS: BALANCING STAKES & CREATIVITY

MORE CONSEQUENCES FOR WRONG ANSWERS → **LOWER TEMPERATURE** (Less Randomness, Higher Determinism)

MORE CREATIVITY REQUIRED → **HIGHER TEMPERATURE** (More Randomness, Higher Novelty)

BUCKET 1: HIGH STAKES & ZERO TOLERANCE

TEMP: 0.0 – 0.2

Zero Tolerance for Surprises

WHY: Creative answers are DANGEROUS. Accuracy is paramount. Hallucinations lead to failure. Need identical output every time.

- Legal contract analyser
- Financial fraud detector
- Medical diagnosis assistant
- Compliance report generator
- Code review agent

BUCKET 2: HELPFUL & CONVERSATIONAL

TEMP: 0.4 – 0.6

Needs Some Personality with Guardrails

WHY: Must feel human, not robotic. Tone matters. Engaging but cannot go off-script or make facts up.

- Customer support agent
- HR onboarding bot
- Sales assistant
- Meeting summariser

BUCKET 3: CREATIVITY IS THE PRODUCT

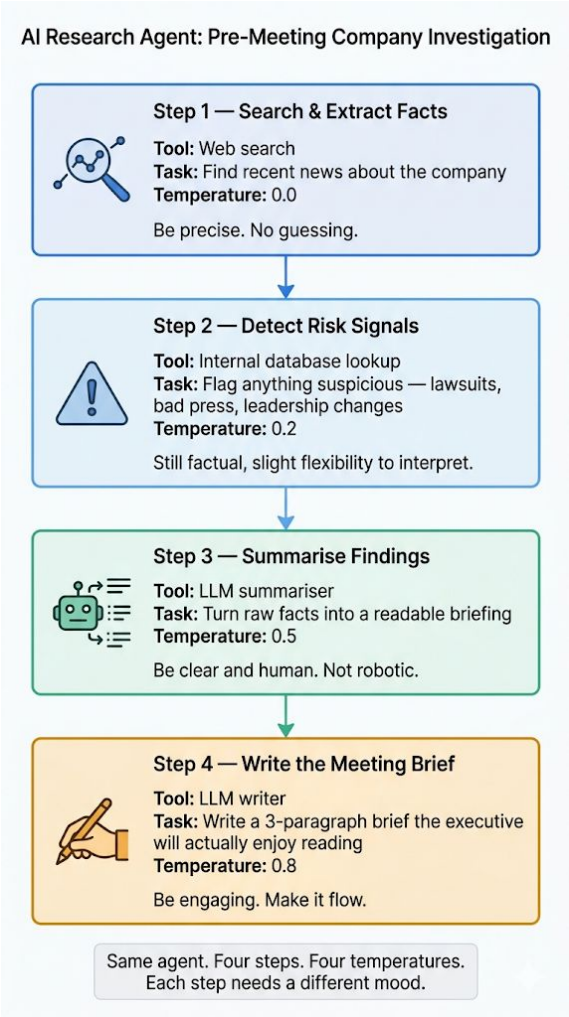
TEMP: 0.7 – 1.0

Surprise and Variety are Key Features

WHY: Novelty and unexpected ideas are essential. Generating diverse outputs is a feature, not a bug. Explores possibilities.

- Marketing copy generator
- Storytelling agent
- Brainstorming assistant
- Ad tagline writer
- Social media content agent

How temperature flows across an actual Agentic AI System



Let's go back to our Google Collab Notebook

Continue coding, analysing, and collaborating.

Part 2: Prompt Templates

Reusable instruction blueprints with fill-in-the-blank placeholders

💡 Analogy: HR writes offer letters weekly. Instead of rewriting from scratch, they use a template: "Dear {name}, we offer you {role} at {salary}." Just fill in the blanks!

📄 **Template**

```
system: "You are a friendly teacher."
human: "Explain {topic} in simple
terms with an example."
```

→
fill in {topic}

✅ **Actual Prompt Sent**

```
system: "You are a friendly
teacher."
human: "Explain Machine Learning
in simple terms with an
example."
```

THE WHATSAPP ANALOGY

"Hey `{name}`, happy birthday!"

Rahul
Hey Rahul, happy birthday!

Priya
Hey Priya, happy birthday!

Amit
Hey Amit, happy birthday!

One message. Ten people. Just fill in `{name}` each time — everything else stays the same.

SAME IDEA IN LANGCHAIN

TEMPLATE (WRITE ONCE)

```
"Give me 3 tips for a {job_role}."
```

→

RESULTS (RUN MANY TIMES)

- Tips for a Product Manager
- Tips for a Data Analyst
- Tips for a HR Manager

> A prompt template is a message with a blank. You fill in the blank — AI does the rest.

HOW AN AI AGENT USES PROMPT TEMPLATES — STEP BY STEP

1 Read the news article

```
"Summarise this article in 3 points: {article_text}"
```

outputs → summary

summary passed into next template

2 Check for risk

```
"Does this mention financial crime? Article: {summary}"
```

outputs → risk_findings

risk_findings passed into next template

3 Write the compliance report

```
"Write a compliance report for: {risk_findings}"
```

outputs → final report

WHY THIS IS A PM'S JOB, NOT JUST AN ENGINEER'S

PM WITHOUT THIS KNOWLEDGE

"The agent should just figure it out on its own."

PM WITH THIS KNOWLEDGE

"At step 2, pass {summary} into the risk detection prompt."

> Design the templates well → agent is predictable.
> Design them poorly → agent goes rogue.

Let's go back to our Google Collab Notebook

Continue coding, analysing, and collaborating.

Part 3: Chains — The Assembly Line

Connect steps in sequence so the output of one becomes input of the next

💡 Analogy: A Zomato order — you place order → restaurant cooks → delivery picks up → food arrives. Each step feeds the next. A LangChain Chain works exactly like this!

📝 **Prompt Template**

Shape the input

| pipe operator (|)

🤖 **LLM**

AI processes it

| pipe operator (|)

📦 **Output Parser**

Clean the output

```
chain = prompt | llm | output_parser
result
 = chain.invoke({ "job_role": "Product Manager" })
```

SAME LOGIC. TWO DIFFERENT WORLDS.

Real World — Job Application

1. **Candidate submits CV**
   Triggers the whole process
2. **HR screens it**
   Filters & shortlists automatically
3. **Hiring Manager reviews**
   Gets the shortlist, not the raw pile
4. **Interview scheduled**
   Flows from the review decision
5. **Offer letter sent**
   Final output delivered

Each stage hands off automatically.
Nobody asks — "what next?"

Same logic. Different world.

LangChain — Chain

1. **User types their question**
   Triggers the chain
2. **Prompt Template formats it**
   Shapes the input for the LLM
3. **LLM reads & generates answer**
   The AI does the thinking
4. **Output Parser cleans it**
   Converts raw text to neat format
5. **User gets formatted result**
   Clean, structured, ready to use

Set it up once.
Runs automatically every time.

```
chain = prompt | llm | output_parser
```

> A chain is like a job pipeline — each stage automatically hands off to the next, without anyone managing it manually.

LangChain Chain

Job Application Pipeline

Real World

1. Candidate submits CV
2. HR screens it
3. Hiring Manager reviews shortlist
4. Interview is scheduled
5. Offer letter is sent

Each stage hands off automatically.
No one asks — what next?

Same logic.
Different world.

LangChain Chain

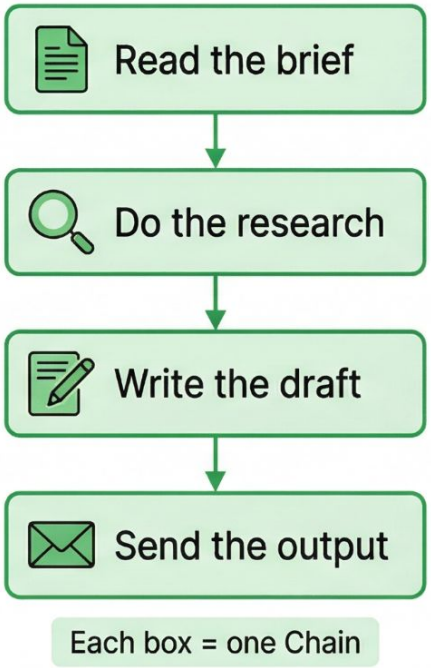
1. User types their question
2. Prompt Template formats it
3. LLM reads and generates answer
4. Output Parser cleans the output
5. User gets a neat, formatted result

Same idea. Code connects each step.
Set it once — runs every time.

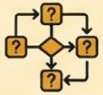
```
chain = prompt | llm | output_parser
```

An AI Agent is nothing but a smart collection of Chains.

What an Agent looks like



What breaks without understanding Chains



Cannot design it properly
You won't know how many steps the agent needs or what each step does.

Cannot debug when it fails
You won't know which step broke or why the answer went wrong.

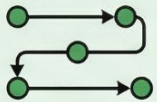
Cannot improve it
Making it better means fixing one step at a time — you need to know where to look.

Cannot control cost and speed
Every Chain = one AI call = time and money. Too many chains = slow and expensive.

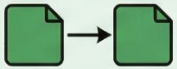
Chains are the building blocks of every AI Agent. Understand them — and you can build anything.

Use a Chain when you know every step in advance. Use an Agent when the AI needs to figure out the steps itself.

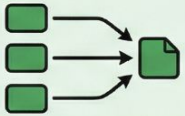
Use a Chain when...



Fixed, predictable steps
You can write down every step before the agent runs.



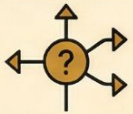
One step feeds the next
Output of step 1 becomes input of step 2. Clean handoff.



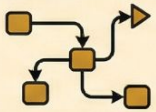
You need consistency
Same steps, same format, every single time.

Task is simple and focused
Lightweight, fast, cheap and easy to debug.

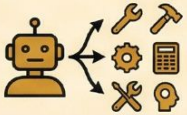
Use an Agent when...



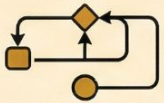
Decision needed mid-way
The system must decide what to do next based on what it found.



Steps are unpredictable
You don't know in advance how many steps are needed.



Tools are used dynamically
The AI picks which tool to use — search, calculator, database — based on the situation.



AI needs to self-correct
If a step goes wrong, the AI evaluates and retries on its own.

Situation	Use		
Fixed steps, predictable flow	Chain	Dynamic, open-ended task	Agent
Decision needed mid-way	Agent	Simple, fast, focused	Chain
Same path every time	Chain	Complex, multi-tool, adaptive	Agent

The LangChain Formula 🎯

Everything we learned today fits into one simple pattern

```
chain = prompt_template | llm | output_parser
```

Concept	What it does	Analogy
LLM	The AI brain that generates text	Smart assistant in a room
Prompt Template	Reusable instructions with placeholders	HR offer letter template
Chain (|)	Connects steps in sequence	Zomato delivery pipeline
Output Parser	Converts AI text to clean format	Formatting a raw reply

Let's go back to our Google Collab Notebook

Continue coding, analysing, and collaborating.

Part 4: Output Parsers

Transform raw AI text into the exact format your app needs

💡 Analogy: You ask: "Top 5 cities in India?" Your assistant replies: "Mumbai, Delhi, Jaipur, Goa, Varanasi." An Output Parser turns that into a proper Python list — automatically!

📄 StrOutputParser

When to use:

Returns plain text — most common, works for everything

```
"Here are 3 tips for PMs: ..."
```

📋 CommaSeparatedListOutputParser

When to use:

Returns a Python list you can loop through

```
["tip 1", "tip 2", "tip 3"]
```

An Output Parser takes a messy reply and converts it something clean, structtured and usable.

Raw replies from colleagues → **Output Parser** → **Structured output**

1. **Aryan:** “I’m free anytime Tuesday.”
2. **Meera:** “Tuesday works, maybe 3pm?”
3. **Kabir:** “Not Monday, Tuesday is fine.”

Raw text in →

Output Parser

Cleans, structures, standardises

→ Clean data out

1	**Name:** Aryan	**Day:** Tuesday	**Time:** Any
2	**Name:** Meera	**Day:** Tuesday	**Time:** 3pm
3	**Name:** Kabir	**Day:** Tuesday	**Time:** Any

Messy. Unstructured. Hard to use.

Clean. Structured. Ready to use.

The AI gives you raw text. The Output Parser gives you something your app can actually use.

In an agent, AI steps talk to each other — not to humans. Output Parsers speak the language that machines understand.

Without Output Parser

AI generates answer → passes messy paragraph → Next step receives it → can't understand it → Agent breaks

Inconsistent. Unpredictable. Breaks silently.

The only difference — one Output Parser.

With Output Parser

AI generates answer → passes to parser → **Output Parser** cleans and structures → passes clean data → Next step receives it → works perfectly → Agent flows

Consistent. Structured. Reliable every time.

What breaks without Output Parsers

- Steps can't talk to each other
- Can't connect to other systems
- Inconsistent behaviour
- Debugging is a nightmare
- Nothing can be built on top

If a machine reads the output next — use a parser.
If a human reads it next — you probably don't need one.

Use an Output Parser when...

Output feeds another agent step.
The next step needs clean, predictable data — not a paragraph.

Saving to a database.
Databases need fields, not sentences.

Displaying in a dashboard or UI.
Tables and charts need rows and columns.

Consistency across runs matters.
Same format every single time, no surprises.

Connecting to an API or tool.
Slack, Jira, Salesforce — they all speak JSON.

Skip the Output Parser when...

Human reads the reply directly.
People read paragraphs. They don't need JSON.

Early prototype or testing.
Just check if the answer is reasonable first.

Creative or open-ended output.
Poems, stories, brainstorming — no fixed structure exists.

Single step, nothing downstream.
If nothing uses the output next, a parser adds zero value.

Situation	Use Parser?		
Output feeds another agent step	Yes	Human reads the reply directly	No
Saving to a database	Yes	Early prototype, just testing	No
Displaying in a dashboard	Yes	Creative or open-ended output	No
Connecting to an API or tool	Yes	Single step, nothing downstream	No

Let's go back to our Google Collab Notebook

Continue coding, analysing, and collaborating.

✏️ Exercise — Your Turn!

Open your Google Colab notebook and complete the exercise

🎯 **Goal: Build a chain that takes a job role → returns 3 career tips in bullet points**

1. **Create a ChatPromptTemplate**
   Use a system + human message. Add {job_role} as a placeholder.
2. **Create a StrOutputParser**
   This converts AI output to plain text.
3. **Build the chain using |**
   Connect: prompt | llm | parser
4. **Run it with .invoke()**
   Pass {"job_role": "your role here"}

🌟 Bonus: Add a {years_of_experience} placeholder and get tips specific to experience level!

🎉 You built your first LangChain app!

What's coming in future sessions:

🧠 Memory

Make AI remember your previous messages — like a conversation that doesn't reset

📄 RAG

Make AI answer questions from YOUR own documents and files

🤖 Agents

Give AI tools — let it search the web, do calculations, and take actions

Questions? 🙋